

Lucrarea nr. 6

1. Scopul lucrării

Lucrarea are ca scop studiul, respectiv proiectarea filtrelor IIR (*Infinite Impulse Response*) cu ajutorul programelor MATLAB și dsPIC FD Lite. Se va realiza și implementarea filtrelor pe dsPIC33FJ256GP506.

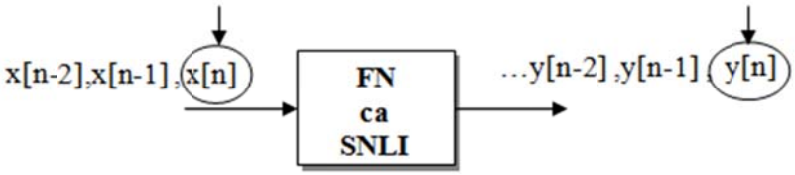
2. Noțiuni teoretice

Filtrul numeric este un sistem (în timp) discret caracterizat printr-un *algoritm numeric* cu ajutorul căruia o secvență de intrare se transformă într-o secvență (filtrată) de ieșire.

Algoritmul (de filtrare) poate să reprezinte: o filtrare trece jos, trece sus, trece bandă, oprește bandă, sau o operație de diferențiere, integrare etc.

2.1. Funcția de transfer a unui filtru numeric

Să considerăm schema generală de filtrare a unei secvențe de intrare $x[n]$ într-o secvență de ieșire $y[n]$:



Un FN poate fi caracterizat de ecuația în timp discret:

$$a_0 y[n] + a_1 y[n-1] + a_2 y[n-1] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2]$$

Dacă $a_0 = 1$, rezultă că secvența de ieșire este dată de relația:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-1]$$

sau:

$$y_n = b_0 x_n + b_1 x_{n-1} + b_2 x_{n-2} - a_1 y_{n-1} - a_2 y_{n-2}$$

Aplicând transformata *Z* ambilor membri rezultă că:

$$Y(z) = b_0 X(z) + b_1 z^{-1} X(z) + b_2 z^{-2} X(z) - a_1 z^{-1} Y(z) - a_2 z^{-2} Y(z)$$

Funcția de transfer a filtrului numeric considerat are expresia:

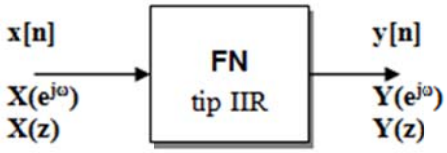
$$H(z)=\frac{Y(z)}{X(z)}=\frac{b_0+b_1z^{-1}+b_2z^{-2}}{1+a_1z^{-1}+a_2z^{-2}}$$

2.2. Avantajele filtrelor numerice

- flexibilitate prin modificarea algoritmului;
- ușor de proiectat, testat și implementat pe un microprocesor sau DSP;
- stabilitate ridicată în raport cu timpul și temperatura;
- siguranță în funcționare, dată de tehnologiile integrate;
- modularitate;
- versatilitate în prelucrarea semnalelor;
- aplicabilitate în joasă frecvență și radiofrecvență.

3. FN cu răspuns infinit la impuls

Să considerăm schema generală a unui filtru numeric de tip IIR:



În timp discret, putem nota:

$$y[n]=b_0x[n]+b_1x[n-1]+...+b_Mx[n-M]-a_1y[n-1]-...-a_Ny[n-N]$$

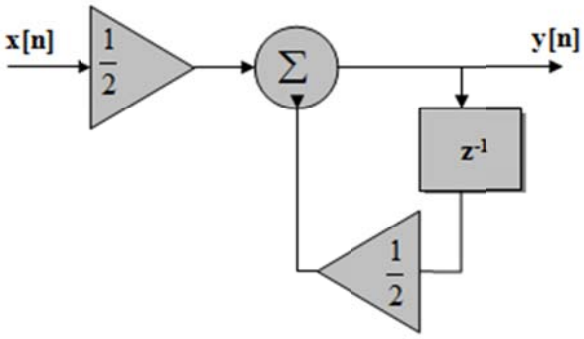
În planul variabilei *z*, funcția de transfer a FN de tip FIR este:

$$H_{FIR}(z)=\frac{Y(z)}{X(z)}=\frac{Z\{y[n]\}}{Z\{x[n]\}}=\frac{\sum_{i=0}^{M}b_iz^{-i}}{1+\sum_{i=0}^{N}a_iz^{-i}}$$

Răspunsul pondere $h_{IIR}[n]$ și funcția de transfer $H_{IIR}(z)$ se obțin prin aplicarea transformatei *Z* directe/inversă:

$$h_{IIR}[n]\overset{Z}{\leftrightarrow}H_{IIR}(z)$$

Exemplu de FN de tip IIR



Rezultă că $y[n]=\frac{1}{2}x[n]+\frac{1}{2}y[n-1]$, astfel că $Y(z)=\frac{1}{2}X(z)+\frac{1}{2}z^{-1}Y(z)$.

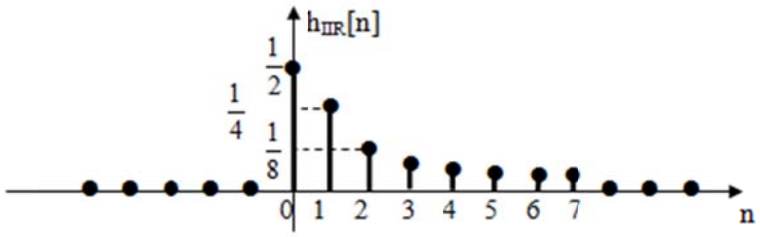
Funcția de transfer a FN-IIR din figură are expresia:

$$H_{FIR}(z)=\frac{Y(z)}{X(z)}=\frac{\frac{1}{2}}{1-\frac{1}{2}z^{-1}}=\frac{1}{2-z^{-1}}$$

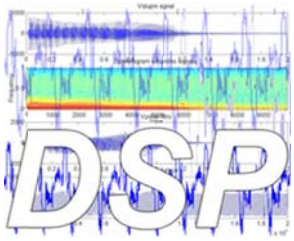
iar răspunsul pondere este:

$$h_{IIR}[n]=Z^{-1}\{H_{IIR}(z)\}=Z^{-1}\left\{\frac{\frac{1}{2}}{1-\frac{1}{2}z^{-1}}\right\}=\frac{1}{2}\left(\frac{1}{2}\right)^{n}\cdot u[n]=\left(\frac{1}{2}\right)^{n+1}\cdot u[n]$$

Reprezentarea grafică a răspunsului pondere este:







3.1. Proiectarea FN-IIR prin metoda aproximării numerice a ecuației diferențiale ce caracterizează un FA

Plecând de la funcția de transfer a FA, se obține, prin transformare de frecvență, funcția de transfer $H_{IIR}(z)$ a FN:

$$H_a(s)\Big|_{s\to\frac{1-z^{-1}}{T}} \to H_{IIR}(z)$$

3.2. Proiectarea FN-IIR prin metoda invarianței la impuls unitate

Algoritm:

1. Se calculează $H_a(s)$ a FA prototip;
2. Se determină $h_a(t) = TL^{-1}\{H_a(s)\}$;
3. Se determină răspunsul pondere a FN astfel încât: $h_a(nT)\big|_{T=1} = h[n]$;
4. Se calculează funcția de transfer a FN: $H_d(z) = Z\{h[n]\}$.

3.3. Proiectarea FN-IIR prin metoda transformării biliniare

Algoritm:

1. Se determină $H_a(s)$ a FA prototip;
2. Se calculează caracteristica de frecvență a FA:

$$|H_a(j\Omega)| \text{ sau } a(\Omega) = 20\lg|H_a(j\Omega)|^{-1}$$

3. Se determină $H_{IIR}(s)$ a FN utilizând transformarea biliniară:

$$H_{IIR}(z) = H_a(s)\Big|_{s\to\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}}$$

4. Se calculează caracteristica de frecvență a FN, care se compară cu cea a FA prototip (și cu datele prescrise FN).

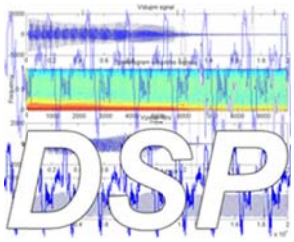
3.4. Proiectarea FN-IIR prin metode de optimizare

Fie funcția de transfer a unui FN-IIR de ordinul întâi:

$$H(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}$$

Valorile parametrilor b_0 ,b_1 ,a_1 se vor determina prin minimizarea funcției criteriu global:





$$\sum_{m=1}^{NF}\left\{\left|H(e^{j\omega_m})\right|-\left|H_{dorit}(e^{j\omega_m})\right|\right\}^2=\min$$

3.5. Aplicații FN-IIR în MATLAB

Comenzi MATLAB pentru proiectarea FN-IIR

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{butter}(n,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{cheby1}(n,Rp,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{cheby2}(n,Rp,Rs,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{ellip}(n,Rp,Rs,Wn,options)$$

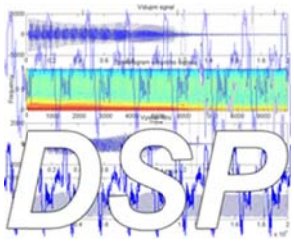
unde $[b,a]$ sunt coeficienții funcției de transfer, iar $[z, p, k]$ sunt zerourile (z), polii (p), respectiv amplificarea (k).

Funcții MATLAB pentru determinarea ordinului (n) și frecvența de tăiere (W_n) pentru un filtru:

$$[n,Wn]=\left\{\begin{matrix}buttord\\cheb1ord\\cheb2ord\\ellipord\end{matrix}\right\}(Wp,Ws,Rp,Rs)$$

unde:

- W_p este frecvența de tăiere pentru banda de trecere; este un scalar sau un vector de două componente cu valorile cuprinse între 0 și 1, unde 1 corespunde frecvenței Nyquist;
- W_s este frecvența de tăiere pentru banda de oprire; este un scalar sau un vector de două componente cu valorile cuprinse între 0 și 1, unde 1 corespunde frecvenței Nyquist;
- R_p – riplul maxim în banda de trecere în *dB*;
- R_s – atenuarea minimă în banda de oprire în *dB*.



Exemplu de proiectare a unui FA prototip

Proiectați un filtru analogic trece bandă (FA-FTB) Chebyshev tip I, de ordinul $n=10$ cu $R_P=3dB$, $F_1=0,1$ ($\Omega_1=2\pi F_1=0,2\pi$) și $F_2=0,5$ ($\Omega_2=\pi$).

```
% determinarea coeficientilor functiei de transfer
% a FTJ analogic cu frecventa de taiere unitate
[b,a]=cheby1(10,3,1,'s');
% reprezentare grafica a modulului si fazei
figure(1);
freqs(b,a);
w1=0.2*pi;w2=pi;Bw=w2-w1;w0=sqrt(w1*w2);
% translatare de frecventa (vezi HELP lp2bp)
[bt,at]=lp2bp(b,a,w0,Bw);
w=linspace(0.01,1,500)*2*pi;
H=freqs(bt,at,w);
figure(2);
% reprezentare grafica a modulului functiei de transfer
semilogy(w/2/pi,abs(H)); grid;
title('Caracteristica de transfer');
xlabel('Frecventa normata');
ylabel('Amplitudine');
```

Proiectarea FN prin aproximare în planul z

a) Proiectați un FN-FIR de tip trece bandă cu aproximare Butterworth, care să satisfacă următoarele specificații:

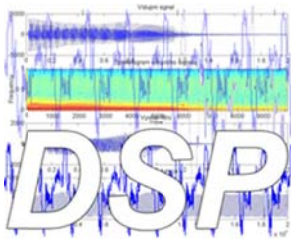
- $R_P=1dB$ pentru $300<f<500Hz$;
- $A_s=20dB$ pentru $f<200Hz$ și $f>650Hz$;
- frecvența de eșantionare este $f_s=1600\text{Hz}$;

Alegeți cel mai mic grad care satisface aceste condiții.

Secvența MATLAB:

```
Fs=1600; %frecventa de esantionare
Fn=Fs/2; %frecventa Nyquist
%frecventele normate
Wp = [300 500]/Fn; Ws = [200 650]/Fn;
Rp = 1; Rs = 20;
%determinare ordin minim filtru
[n,Wn] = buttord(Wp,Ws,Rp,Rs);
%determinarea coeficientilor filtrului
[b,a] = butter(n,Wn);
```

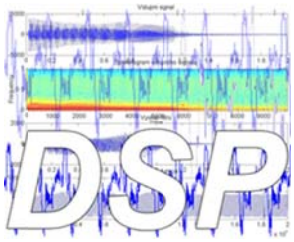





```
%raspunsul la impuls
h=impz(b,a);
stem(h);grid
title('Raspunsul la impuls');
%caracteristici de transfer
figure
[H,W]=freqz(b,a);
subplot(211)
plot(W/(2*pi)*Fs, abs(H));grid
title('Caracteristica de transfer');
xlabel('Frecventa [Hz]');
ylabel('Amplitudine')
subplot(212)
plot(W/(2*pi)*Fs, unwrap(angle(H))*180/pi);grid
title('Caracteristica de faza');
xlabel('Frecventa [Hz]');
ylabel('Faza [grade]');
```

Proiectați un FN tip FTB cu banda de trecere $[1000,2000]Hz$, benzile de blocare $[0,500]Hz$ și $[2500,-]Hz$, iar $R_P = 1dB$, $R_S = 60dB$, cu $f_e = 10kHz$.

```
fe=10e3; %frecventa de esantionare
fn=fe/2; %frecventa Nyquist
Wp=[1000, 2000]/fn; %banda de trecere
Ws=[500,2500]/fn; %frecventele de taiere
Rp=1; Rs=60;
% determinarea ordinului filtrului
% si frecventa de taiere
[n,Wn]=buttord(Wp, Ws, Rp, Rs);
%calculul coeficientilor filtrului
[b,a]=butter(n,Wn);
% (sau:=)
%[n,Wn]=ellipord(Wp, Ws, Rp, Rs)
%[b,a]=ellip(n,Rp,Rs,Wn)
[H,W]=freqz(b,a);
subplot(211);
plot(W/(2*pi), abs(H));grid;
title('Caracteristica de transfer');
xlabel('Frecventa normata');
ylabel('Amplitudine');
subplot(212);
plot(W/(2*pi), unwrap(angle(H))*180/pi);grid;
title('Caracteristica de faza');
xlabel('Frecventa normata');
ylabel('Faza [grade]');
```



Să se ruleze programul și pentru filtru de tip eliptic și să se precizeze diferențele.

Proiectarea FN-IIR prin metoda invarianței la impuls

Să se proiecteze un filtru FTB discret de tip Chebyshev I, prin metoda invarianței la impuls, având riplul în banda de trecere de maxim $3dB$, atenuarea în banda de oprire de minim $20dB$, $F_s = 2$, iar frecvențele corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,3,\ F_{p2} = 0,4,\ F_{s1} = 0,2,\ F_{s2} = 0,5.$$

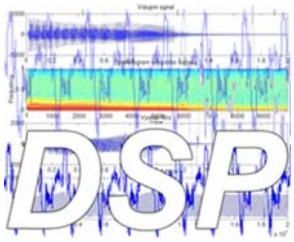
Se folosește funcția `[numd,dend]=impinvar(num,den,Fs)`, care creează un filtru digital având coeficienții numărătorului și numitorului returnați în vectorii *numd*, respectiv *dend*. Răspunsul la impuls al filtrului numeric se obține prin eșantionarea filtrului analogic, ai cărui coeficienți sunt conținuți în vectorii *num*, respectiv *den*, cu frecvența de eșantionare *Fs* (*Hz*). Dacă nu se precizează *Fs* atunci aceasta se ia implicit $1Hz$. Această funcție se aplică numai pentru filtre descrise prin funcția de transfer. Metoda invarianței răspunsului la impuls nu este adecvată pentru FTS și FOB.

Secvența MATLAB:

```
Rp=3;Rs=20;
Wp=2*pi*[0.3 0.4];
Ws=2*pi*[0.2 0.5];
% calculul ordinului si a frecventei de taiere
% a filtrului analogic
[n,Wn]=cheb1ord(Wp,Ws,Rp,Rs,'s');
% proiectarea filtrului analogic prototip
[z,p,k]=cheb1ap(n,Rp); %vezi help cheb1ap
% determinare coeficienti functie de transfer
[num,den]=zp2tf(z,p,k); %vezi help zp2tf
Wo=sqrt(Wn(1)*Wn(2));
Bw=Wn(2)-Wn(1);
% translatare de frecventa (vezi HELP lp2bp)
[numt,dent]=lp2bp(num,den,Wo,Bw);
w=linspace(0,1,500)*2*pi;
H=freqs(numt,dent,w);
plot(w/2/pi,20*log10(abs(H)), 'r');
ylabel('Amplitudine (dB)');
hold on;

% proiectare FTB discret prin metoda
% invariantei la impuls Fs=2
[bz,az]=impinvar(numt,dent,2);
```






```
[Hd,Wd]=freqz(bz,az);
plot(Wd/(pi), 20*log10(abs(Hd)), 'b'); grid;
xlabel('frecventa normata la frecventa Nyquist');
```

Proiectarea FN-IIR prin transformarea biliniară

Să se proiecteze un filtru FTB discret de tip Chebyshev I, prin metoda transformării biliniare, având riplul în banda de trecere de maxim $3dB$, atenuarea în banda de oprire de minim $20dB$, iar frecvențele discrete corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,25,\ F_{s1} = 0,15,\ F_{s2} = 0,3$$

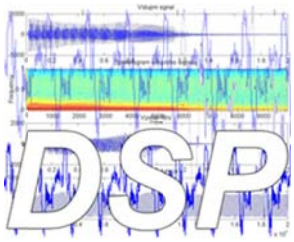
Frecvența de eșantionare este $F_S = 2$.

Se folosește funcția `[numd,dend]=bilinear(num,den,Fs)`, unde:

- *num* și *den* sunt vectori linie ce conțin coeficienții funcției de transfer ai filtrului analogic;
- *numd* și *dend* sunt coeficienții funcției de transfer ai filtrului digital.

Secvența MATLAB:

```
wp=2*pi*[0.1 0.15];
ws=2*pi*[0.05 0.2];
Rp=3;Rs=20;Fs=2;
%specificarea parametrilor filtrului analogic
Wp=wp*Fs;
Ws=ws*Fs;
%calculul ordinului si a frecventei de
%taiere a filtrului analogic
[n,Wn]=cheb1ord(Wp,Ws,Rp,Rs,'s');
%proiectarea filtrului analogic prototip
[z,p,k]=cheb1ap(n,Rp);
[num,den]=zp2tf(z,p,k); %vezi help zp2tf
%translarea de frecventa
Wo=sqrt(Wn(1)*Wn(2));
Bw=Wn(2)-Wn(1);
[numt,dent]=lp2bp(num,den,Wo,Bw);
%raspunsul in frecventa al filtrului analogic
w=linspace(0,1,500)*2*pi;
h=freqs(numt,dent,w);
%proiectarea filtrului digital Fs=2
[bz,az]=bilinear(numt,dent,Fs);
%raspunsul in frecventa al filtrului
[hd,wd]=freqz(bz,az);
%reprezentarea grafica
figure(1);
```




```
plot(wd/pi,20*log10(abs(hd)),'r',w/2/pi,20*log10(abs(h)
),'b'); grid;
xlabel('F (Hz) pt analogic \omega/\pi pt. discret');
ylabel('Amplitudine (db)');
legend('filtru discret','filtru analogic');
```

După cum era de aşteptat, frecvenţa benzii de trecere şi a benzii de oprire ale filtrului discret nu coincid cu cele specificate. Acest lucru se datorează neliniarităţii transformării biliniare.

Proiectarea FN-IIR prin optimizarea în planul *z*

Proiectarea unui FN-IIR care aproximează în sensul erorilor pătratice minime o caracteristică de frecvenţă foloseşte funcţia:

```
[b,a]=yulewalk(n,f,m)
```

unde:

m - un vector cu valorile amplitudinilor dorite;
f – un vector cu valorile frecvenţelor specificate.

Exemplu: Proiectaţi un FN de gradul $n=8$ de tip trece jos, a cărui caracteristică de frecvenţă este specificată prin:

```
f=[0 0.6 0.6 1]; m=[1 1 0 0 ];
[b,a]=yulewalk(8,f,m);[h,w]=freqz(b,a);
plot(f,m,'r',w/pi,abs(h),'b'); grid;
legend('caracteristica dorita', 'caracteristica
filtrului proiectat');
xlabel('Frecventa normata la frecventa Nyquist');
ylabel('Castig');
```

Implementarea unui filtru IIR pe dsPIC33FJ256GP506

Relaţia în timp discret care caracterizează structura unui filtru FIR este:

$$y[n]=\sum_{i=0}^{M} b_i x[n-i]-\sum_{j=0}^{N} a_j y[n-j]$$

unde:

$y[n]$ – este semnalul de la ieşirea filtrului;

$x[n]$ – este semnalul de la intrarea filtrului;

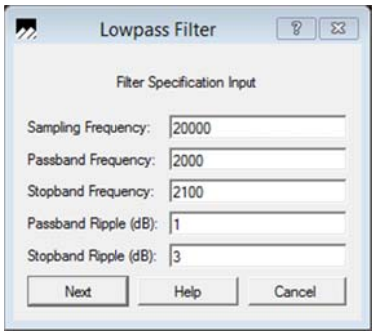
b_i – sunt coeficienţii numărătorului funcţiei de transfer;

a_j – este coeficienţii numitorului funcţiei de transfer.

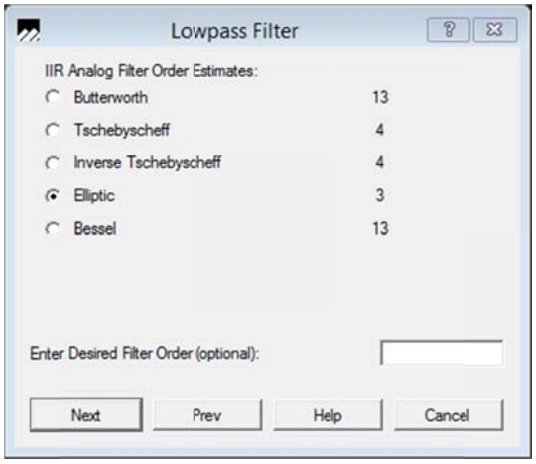
Așa cum se poate observa, eșantionul semnalului de la ieșire la momentul de timp curent se calculează prin combinația liniară a eșantioanelor semnalului de la intrare. Deci, la fiecare moment de timp, trebuie calculată o sumă de produse, adică trebuie executate *M+N+1* înmulțiri și *M+N-1* adunări și o scădere.

Se va implementa un filtru trece jos IIR în formă transpusă, având frecvența de tăiere la 1*kHz*. Se va proiecta filtrul în dsPIC FD Lite.

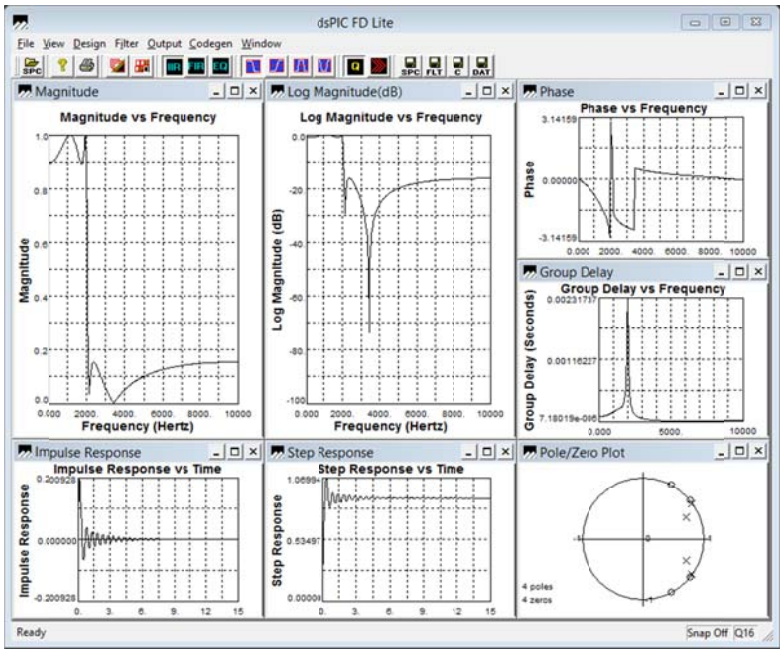
În continuare este prezentată procedura de proiectare a unui filtru folosind programul dsPIC FD Lite. După deschidere, se urmează secvența: *Design → IIR Design → Lowpass* și se introduc datele, ca în figura de mai jos:



Se alege tipul filtrului și/sau se poate specifica ordinul său. Am ales un filtru eliptic, de ordin 4, aplicația fiind limitată la proiectarea filtrelor IIR de ordin maxim 4.



Următoarea fereastră prezintă grafic răspunsul filtrului şi variaţia altor parametri în funcţie de frecvenţă:



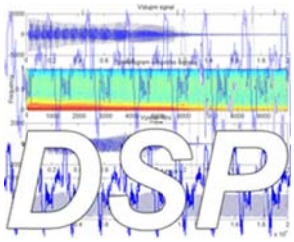
Pentru generarea fişierului cu coeficienţi se selectează: *Codegen → Microchip → dsPIC30 → Program Space →* se debifează *C Header File and Sample Calling Sequence (.h)* şi *C Wrapper*. Se va salva fişierul cu denumirea *lowpass_psv*. Dacă coeficienţii ar fi fost stocaţi în RAM, se alegea denumirea *lowpass*. În mod asemănător, se procedează şi pentru alte tipuri de filtre, cu menţiunea că se folosesc denumirile *highpass*, *passband*, respectiv *stopband*.

Proiectul va conţine fişierele antet *dsp.h* şi *dspcommon.inc*, şi fişierul sursă *iirtrans.s* şi cel furnizat de programul dsPIC FD Lite. Fişierul sursă se găseşte în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*.

Rutinele scrise în limbaj de asamblare operează cu o structură numită *IIRTransposedStruct*, descrisă în *dsp.h*. Aceasta conţine următoarele variabile:

- *numSectionsLess1* – numărul de secţiuni de ordinul doi cascadate minus una;
- *coeffsBase* – pointer către vectorul de coeficienţi;
- *coeffsPage* - variabilă care indică locaţia în care sunt stocaţi coeficienţii filtrului, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*) sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceştia);





- *delayBase1* – pointer către buffer-ul de întârziere 1;
- *delayBase2* – pointer către buffer-ul de întârziere 2;
- *finalShift* – numărul de biți cu care să se deplaseze biții secvenței de ieșire (implicit este 0; o valoare pozitivă înseamnă deplasare la stânga, iar o valoare negativă înseamnă deplasare la dreapta; această opțiune se folosește pentru scalarea rezultatului, pentru aducerea câștigului filtrului la 1).

În procesul de filtrare trebuie să se țină cont de eșantioanele filtrate și cele care urmează să fie filtrate. Acest lucru se realizează folosind o secvență adițională, numită *delay* (întârziere). Înainte de filtrare, secvența respectivă conține eșantioanele unei secvențe filtrate anterior. După operația de filtrare, secvența conține ultimele eșantioane filtrate. Pentru operare corectă, trebuie ca această secvență să fie inițializată cu 0. Acest lucru se realizează folosind funcția *IIRTransposedInit*, care acceptă ca argument pointer-ul către structura IIR definită cu funcția *IIRTransposedStruct*.

Funcția care realizează efectiv filtrarea este *IIRTransposed*, și are următoarele argumente:

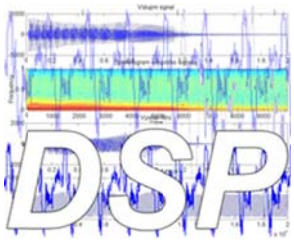
- *numSamps* – numărul de eșantioane de intrare (care trebuie filtrate);
- *dstSamps* – pointer către vectorul de ieșire, ce va conține eșantioanele semnalului rezultat în urma operației de filtrare;
- *srcSamps* – pointer către vectorul de intrare, ce conține eșantioanele semnalului supus filtrării;
- *filter* – pointer către structura IIR definită cu funcția *IIRTransposedStruct*.

Pentru această aplicație se va filtra un semnal dreptunghiular, de frecvență 1*kHz*, eșantionat cu frecvența de 20*kHz*, generat folosind programul dsPICworks. Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locația *D:\Projects\h*, iar în arborele de căutare pentru fișiere scrise în limbaj de asamblare (*Assembler Include Search Path*), locația *D:\Projects\src*.

Programul principal:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog
```



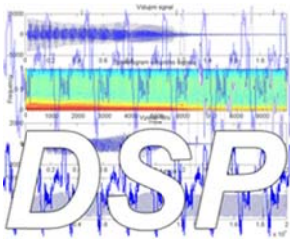

```
#define BLOCK_LENGTH 256 //Marimea secventei care va fi
filtrata
#define LOWPASSFILTER
//#define HIGHPASSFILTER
//#define BANDPASSFILTER
//#define BANDSTOPFILTER
#define FILTERCOEFFS_IN_PROGMEM //Se comenteaza aceasta linie
daca coeficientii filtrului sunt stocati in memoria RAM

//Secventa de intrare - semnal dreptunghiular, de frecventa
1kHz, esantionat cu frecventa de 20kHz
fractional square1k[256] =
{0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001};

//Secventa de iesire (secventa de intrare filtrata)
fractional FilterOut[BLOCK_LENGTH];

#ifdef LOWPASSFILTER
```

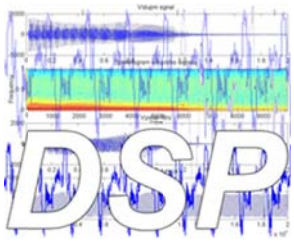





```
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct lowpass_psvFilter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria de program
        #else
                extern IIRTransposedStruct lowpass_Filter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef HIGHPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
highpass_psvFilter; //Structura de filtru trece sus, cu
coeficientii salvati in memoria de program
        #else
                extern IIRTransposedStruct highpass_Filter;
//Structura de filtru trece sus, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
bandpass_psvFilter; //Structura de filtru trece banda, cu
coeficientii salvati in memoria de program
        #else
                extern IIRTransposedStruct bandpass_Filter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDSTOPFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
bandstop_psvFilter; //Structura de filtru opreste banda, cu
coeficientii salvati in memoria de program
        #else
                extern IIRTransposedStruct bandstop_Filter;
//Structura de filtru opreste banda, cu coeficientii salvati
in memoria RAM
        #endif
#endif

int main(void) {

                //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
                //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
```




```
                //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact
intrare de 7.37M
                PLLFBD=41;                      //M=39
                CLKDIVbits.PLLPOST=0;           //N1=2
                CLKDIVbits.PLLPRE=0;            //N2=2
                OSCTUN=0;

                __builtin_write_OSCCONH(0x01);
        //Initializare oscilator la FRC (Fast RC) cu PLL
                __builtin_write_OSCCONL(0x01);
                while (OSCCONbits.COSC != 0b01);
        //Astepta realizarea comutarii
                while(!OSCCONbits.LOCK);

         //Realizarea filtrarii
         #ifdef LOWPASSFILTER
                 #ifdef FILTERCOEFFS_IN_PROGMEM
                 //Initializarea secventei delay

IIRTransposedInit(&lowpass_psvFilter);
                 //Filtrarea secventei de intrare

        IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0]
,&lowpass_psvFilter);
                 #else
                 //Initializarea secventei delay
                         IIRTransposedInit(&lowpass_Filter);
                 //Filtrarea secventei de intrare

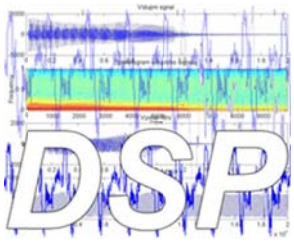
IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpas
sFilter);
                 #endif
                #endif
                #ifdef HIGHPASSFILTER
                 #ifdef FILTERCOEFFS_IN_PROGMEM
                 //Initializarea secventei delay

IIRTransposedInit(&highpass_psvFilter);
                 //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpa
ss_psvFilter);
                 #else
                 //Initializarea secventei delay
                         IIRTransposedInit(&highpass_Filter);
                 //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpa
ssFilter);
```






```
                #endif
               #endif
        #ifdef BANDPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay

IIRTransposedInit(&bandpass_psvFilter);
                //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpa
ss_psvFilter);
                #else
                //Initializarea secventei delay

IIRTransposedFilterInit(&bandpass_Filter);
                                //Filtrarea secventei de
intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpa
ssFilter);
                #endif
               #endif
               #ifdef BANDSTOPFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay

IIRTransposedInit(&bandstop_psvFilter);
                //Filtrarea secventei de intrare

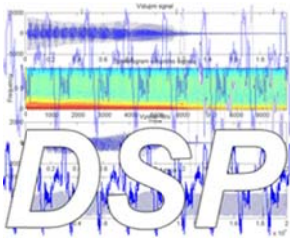
IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandst
op_psvFilter);
                #else
                //Initializarea secventei delay

IIRTransposedFilterInit(&bandstop_Filter);
                //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandst
opFilter);
                #endif
        #endif

               //Bucla de program
        while (1);
}
```

Se vor vizualiza secvențele de intrare și de ieșire.



Programul principal este conceput utilizând directive de preprocesare, astfel încât să permită folosirea tuturor tipurilor de filtre. De exemplu, dacă se dorește utilizarea unui filtru trece bandă, atunci se urmează pașii:

- se proiectează filtrul în dsPIC FD Lite şi se generează fişierul de coeficienți în forma necesară funcțiilor din MPLAB;
- se adaugă fișierul respectiv proiectului;
- se comentează liniile *#define LOWPASSFILTER*, *#define HIGHPASSFILTER* și *#define BANDSTOPFILTER*;
- se scoate de sub comentariu linia *#define BANDPASSFILTER*;
- dacă se dorește salvarea coeficienților în memoria RAM, atunci se comentează și linia *#define FILTERCOEFFS_IN_PROGMEM*.

4. Desfășurarea lucrării

1. Se vor rula în MATLAB toate aplicațiile descrise în breviarul teoretic și se va urmări execuția lor.

2. Proiectaţi un FN-FIR de tip trece bandă cu aproximare Butterworth care să satisfacă următoarele specificaţii:

- $R_p = 3\text{dB}$ pentru $1000 < f < 1300\text{Hz}$;
- $A_s = 30\text{dB}$ pentru $f < 800\text{Hz}$ şi $f > 1500\text{Hz}$;
- frecvenţa de eşantionare este $f_s = 8000\text{Hz}$.

Alegeţi cel mai mic grad care satisface aceste condiţii.

Să se repete proiectarea aceluiaşi filtru pentru aproximare Chebyshev I, II şi eliptic (MATLAB).

3. Să se proiecteze un filtru FTB discret de tip Butterworth prin metoda invarianţei la impuls, având riplul în banda de trecere de maxim 1*dB*, atenuarea în banda de oprire de minim 40*dB*, F_s=2, iar frecvenţele corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,35,\ F_{s1} = 0,1,\ F_{s2} = 0,4$$

4. Să se proiecteze un filtru FTB discret de tip Butterworth prin metoda transformării biliniare, având riplul în banda de trecere de maxim 2*dB*, atenuarea în banda de oprire de minim 30*dB*, frecvenţa de eşantionare este Fs=2, iar frecvenţele discrete corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,35,\ F_{s1} = 0,15,\ F_{s2} = 0,4$$

5. Se va implementa pe dsPIC33FJ256GP506 un filtru notch ce trebuie să anuleze frecvenţa de 1000*Hz*, factorul de calitate Q = 10 şi frecvenţa de eşantionare 8000*kHz*.